

Machine Learning

Week 8 Lecture

Dr. Muhammad Ahmed Raza

Contents

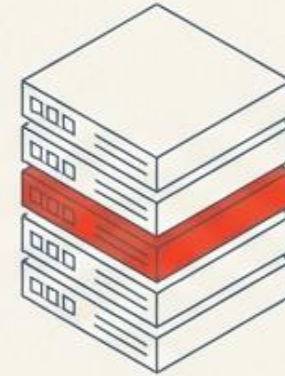
- Intro to Anomaly Detection
- Distance metrics
- Isolation Forests
- The Gaussian Model

Searching for the Exception, Not the Rule

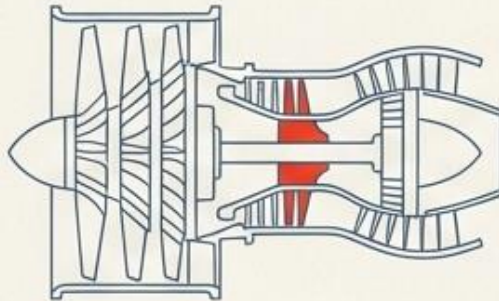
In critical systems, anomalies are rare, structurally different, and disproportionately important. We are no longer profiling the massive bulk of normal data; we are hunting the needle in the haystack.



Fraud Detection



Network Intrusion

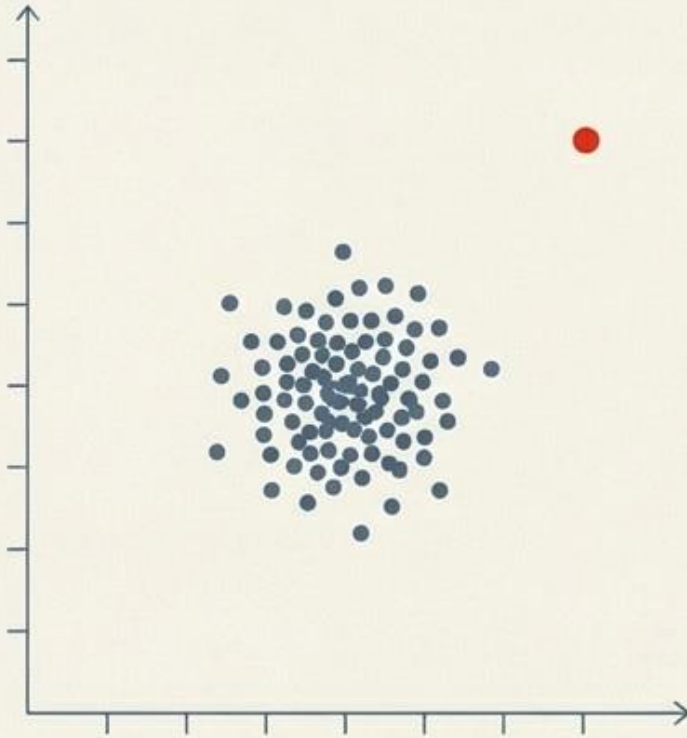


IoT Sensor Faults



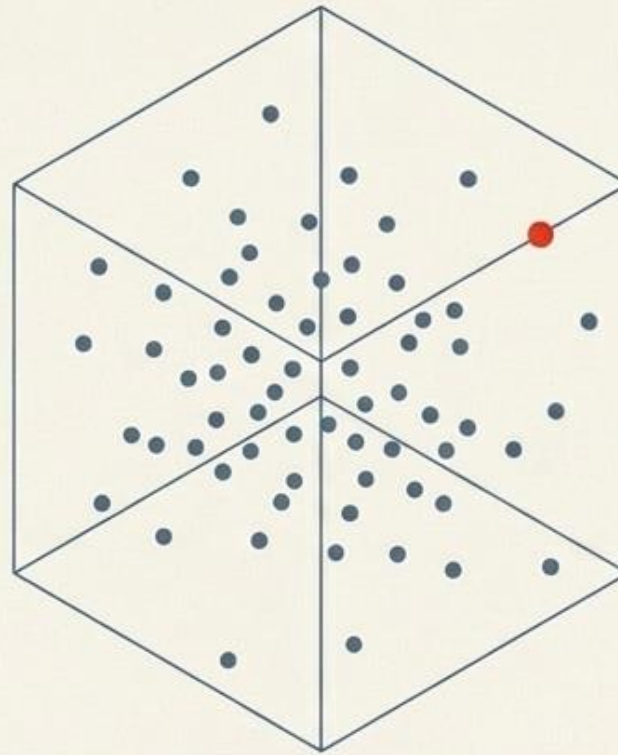
Medical Diagnostics

The Villain: The Curse of Dimensionality



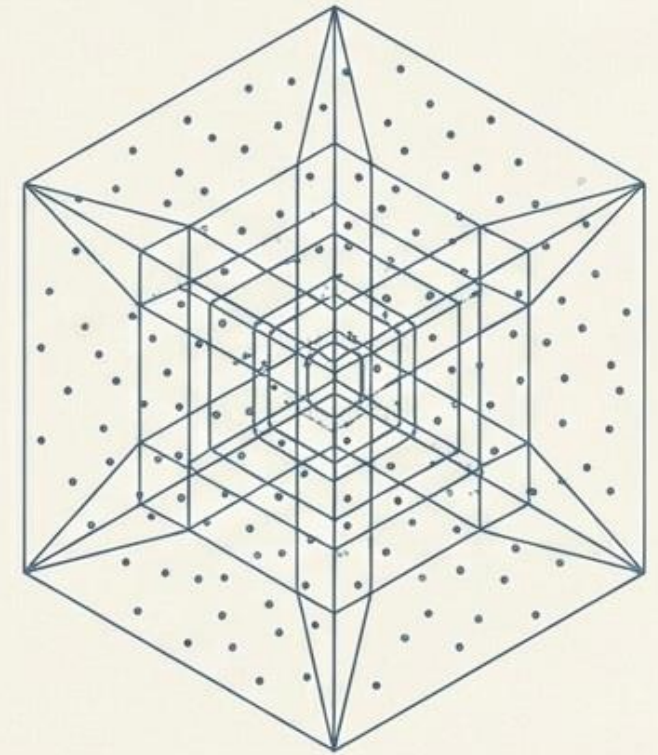
Low Dimensions

Data is dense. Intuitive clustering is visible.



Moderate Dimensions

Data density exponentially decreases. Separation becomes ambiguous.



High Dimensions

The mathematical fabric changes. All points become uniformly sparse.

Distance Metric Failure (Relative Contrast)

- Relative Contrast is a mathematical measure used to determine how effectively a distance metric (like Euclidean distance) can distinguish between the "nearest" and "farthest" points in a dataset.

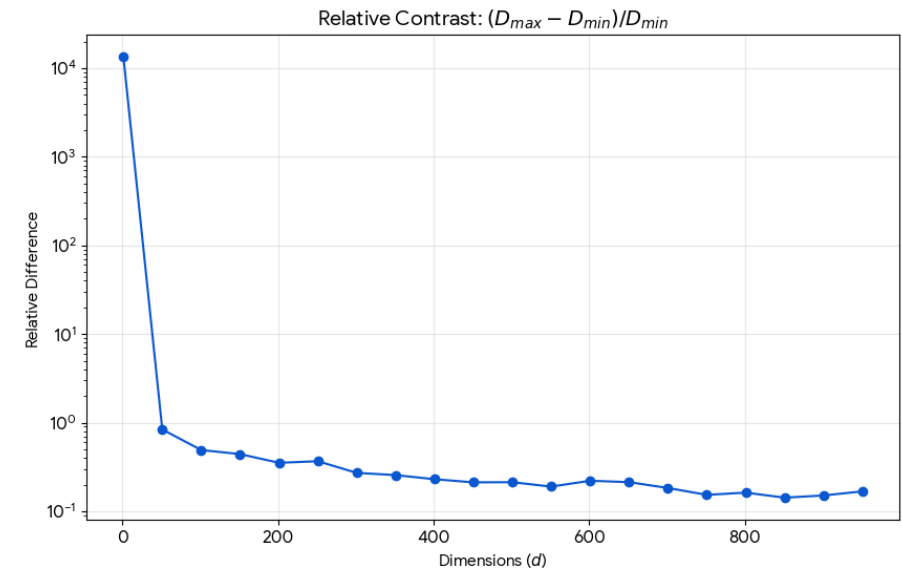
- The Mathematical Formula

- Relative contrast is typically defined as the ratio of the range of distances to the minimum distance:

$$C_r = \frac{D_{max} - D_{min}}{D_{min}}$$

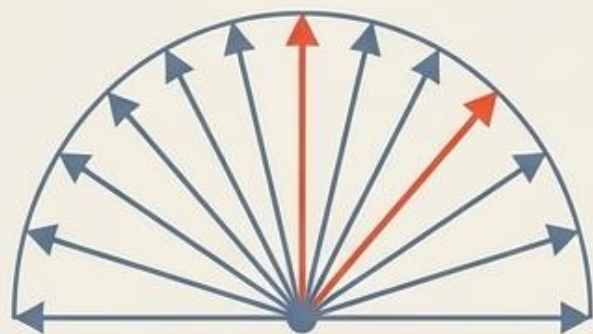
- Where:

- D_{max} : The distance to the farthest neighbor from a query point.
 - D_{min} : The distance to the nearest neighbor from a query point.



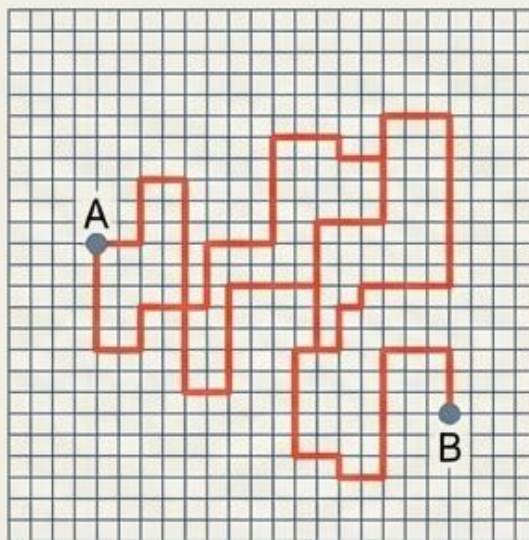
Why Traditional Metrics Break Down

Algorithms relying on spatial proximity are fundamentally unreliable in high-dimensional space due to metric convergence and noise dominance.



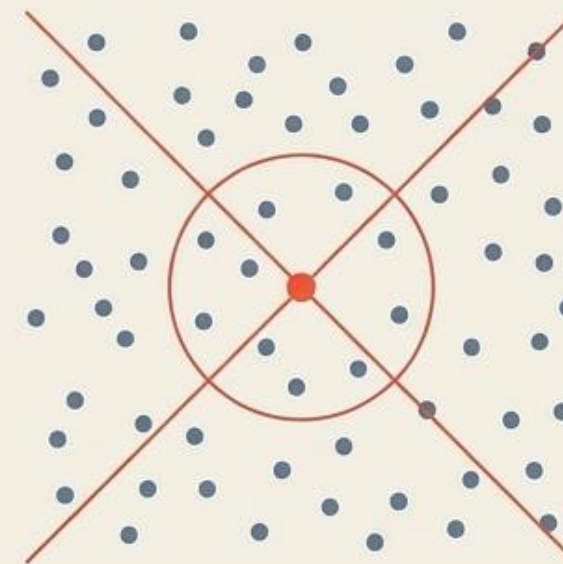
Euclidean Distance

Equidistance Issue. Vectors collapse to the same length. The difference between the nearest and farthest neighbor becomes statistically insignificant.



Manhattan Distance

Noise Dominance. Paths are arbitrarily inflated by irrelevant noise dimensions, washing out the crucial features that actually matter.

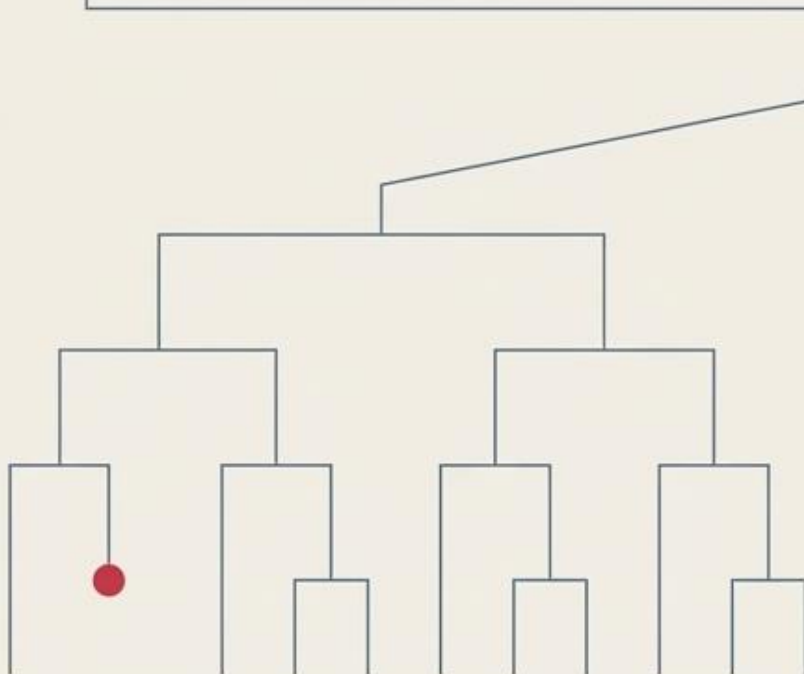


K-Nearest Neighbors

Randomized Neighbors. When all points are practically equidistant, the concept of a "nearest" neighbor becomes effectively random.

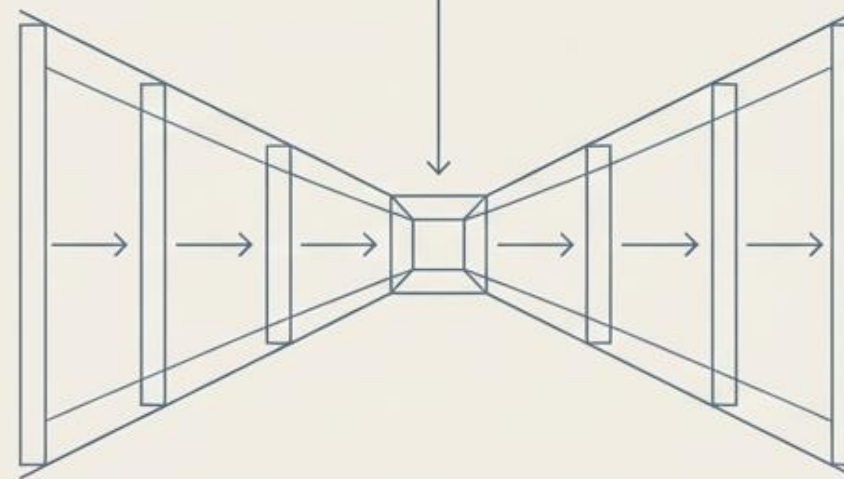
The Paradigm Shift

If we cannot mathematically measure how far an anomaly is from normal, we must abandon distance altogether. We need entirely new philosophies of detection.



Don't measure distance. Isolate directly.

Paradigm 1: Isolation Forests



Don't look for anomalies. Learn normality.

Paradigm 2: Autoencoders or gaussians, etc

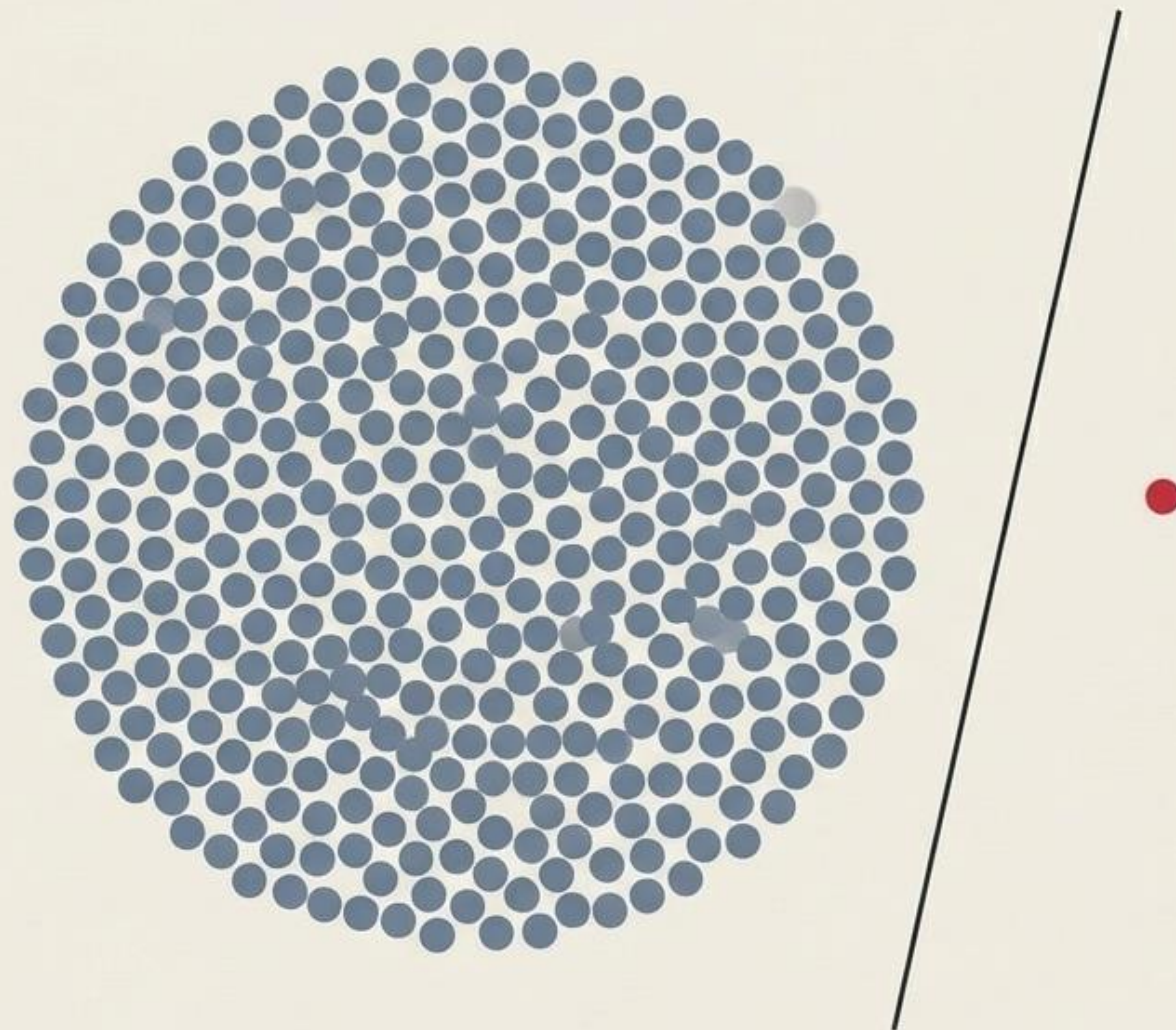
Paradigm 1: Isolation Forests

Core Insight

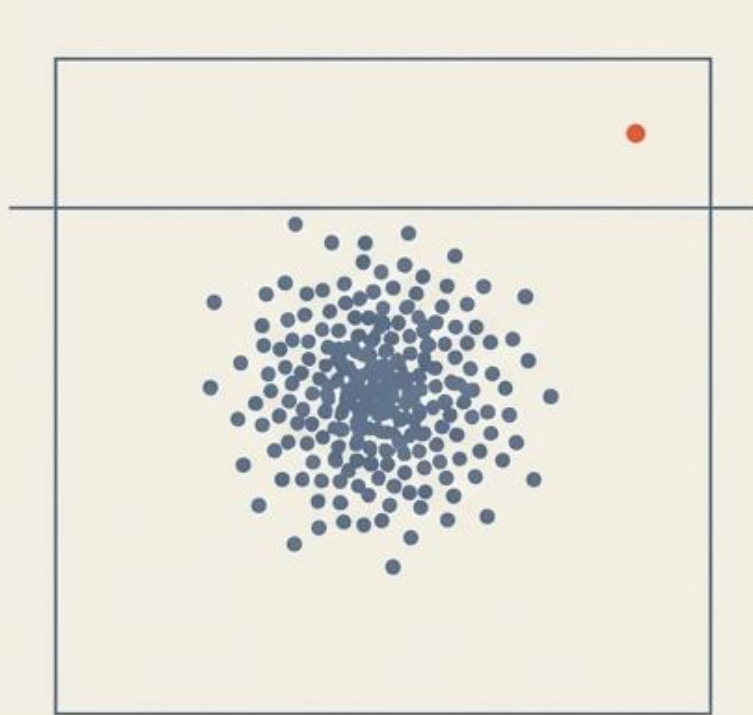
Anomalies possess two distinct properties: they are Few and they are Different.

The Strategy

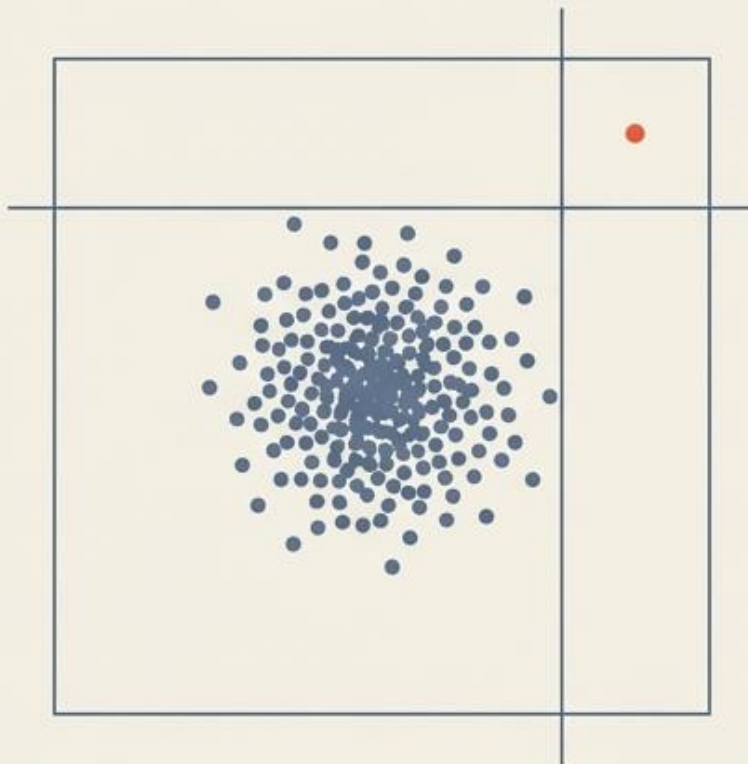
Stop trying to build a complex profile of the massive normal cluster. Instead, build random mathematical fences. Because anomalies are few and structurally different, they get fenced into their own sectors much faster than normal data.



The Geometry of Isolation

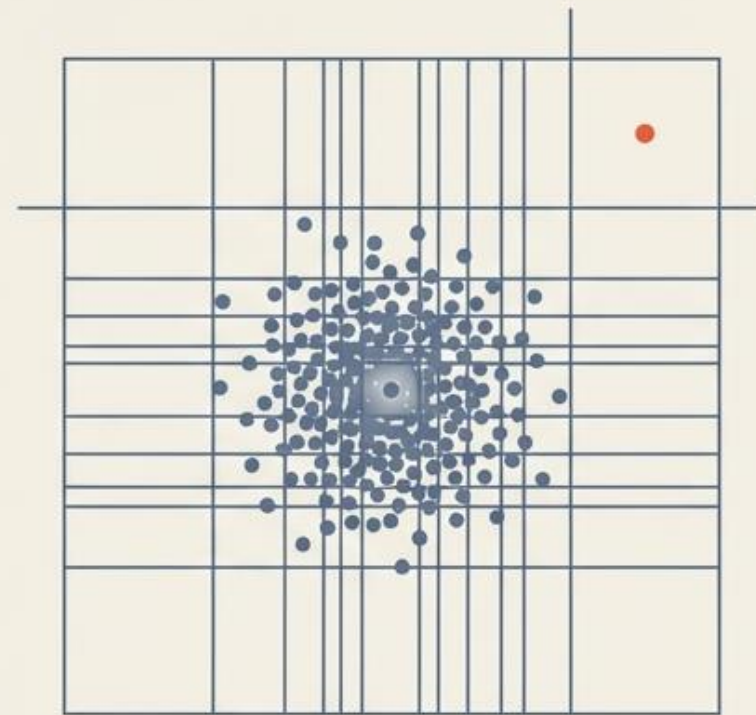


Step 1: Random Cut



Step 2: Anomaly Isolated

Anomalies: Located on spatial edges. Require a very short path length (few cuts) to isolate.



Step 3: Normal Data Isolated

Normal Data: Buried deep in the data manifold. Require a long path length (many overlapping cuts) to finally isolate.

Calculating the Anomaly Score

$$s(x, n) = 2^{-h(x)/c(n)}$$

Path Length (number of cuts to isolate)

Average path length of unsuccessful isolations



Translation

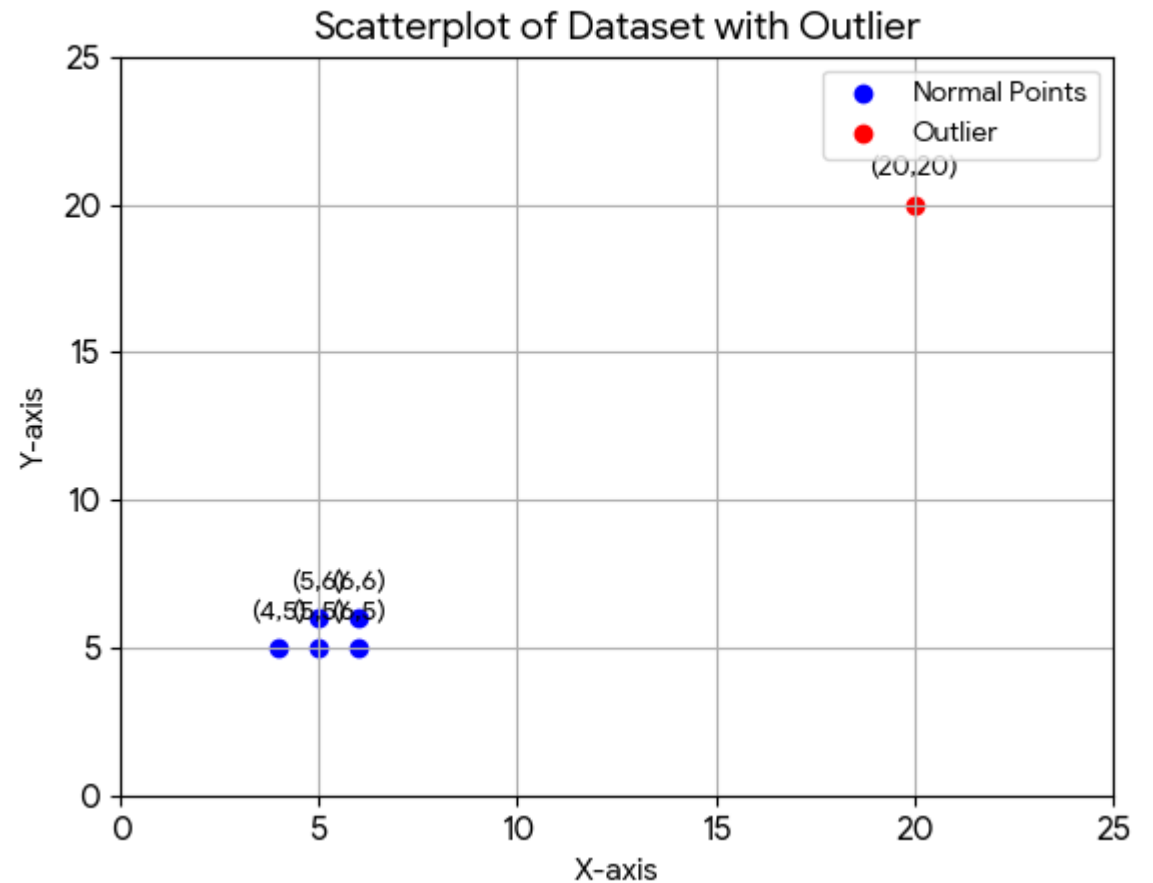
The algorithm translates the number of cuts into a confidence score from 0 to 1. Closer to 1 indicates a high likelihood of being an anomaly.

Verdict

Scales brilliantly and completely bypasses the curse of dimensionality. However, random splits sacrifice some interpretability.

2D Example

- Imagine this dataset:
 - Most points clustered around:
 - (4,5), (5,5), (6,6), (5,6), (6,5)
 - One outlier:
 - (20,20)
- $h(x_{anomaly}) = 1$
 - Number of cuts required
- $h(x_{normal}) = 3$
 - Imagine splitting (5,5)



Scoring

- Normalization Factor (c) Formula:

$$c(n) \approx 2 \cdot H(n-1) - \frac{2(n-1)}{n}$$

For small n , we approximate:

$$H(5) \approx 2.283$$

So:

$$c(6) \approx 2(2.283) - \frac{2 \cdot 5}{6} \approx 4.566 - 1.667 \approx 2.9$$

- Where H is the harmonic number

$$H(5) = 1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \frac{1}{5}$$

$$H(5) = 1 + 0.5 + 0.33 + 0.25 + 0.2 = 2.28$$

Scoring

- Calculation

- Anomaly Score

$$s(x) = 2^{-\frac{h(x)}{c(n)}}$$

$$s(\text{anomaly}) = 2^{-\frac{1}{2.9}} \approx 2^{-0.34} \approx 0.79$$

- Normal Point Score

$$s(\text{normal}) = 2^{-\frac{3}{2.9}} \approx 2^{-1.03} \approx 0.49$$

Summary

- Criteria:
 - Scores greater than 0.5 (anomalous)
 - Scores closer to 1 → anomaly
 - Scores around 0.5 → normal
 - Scores near 0 → very normal

Point	Path Length	Score	Meaning
(20,20)	1	0.79	🚨 Likely anomaly
(5,5)	3	0.49	✅ Normal

Paradigm 2

- Learning Normality
 - The Gaussian Model
 - Autoencoders (not discussed)

Anomaly detection example

Aircraft engine features:

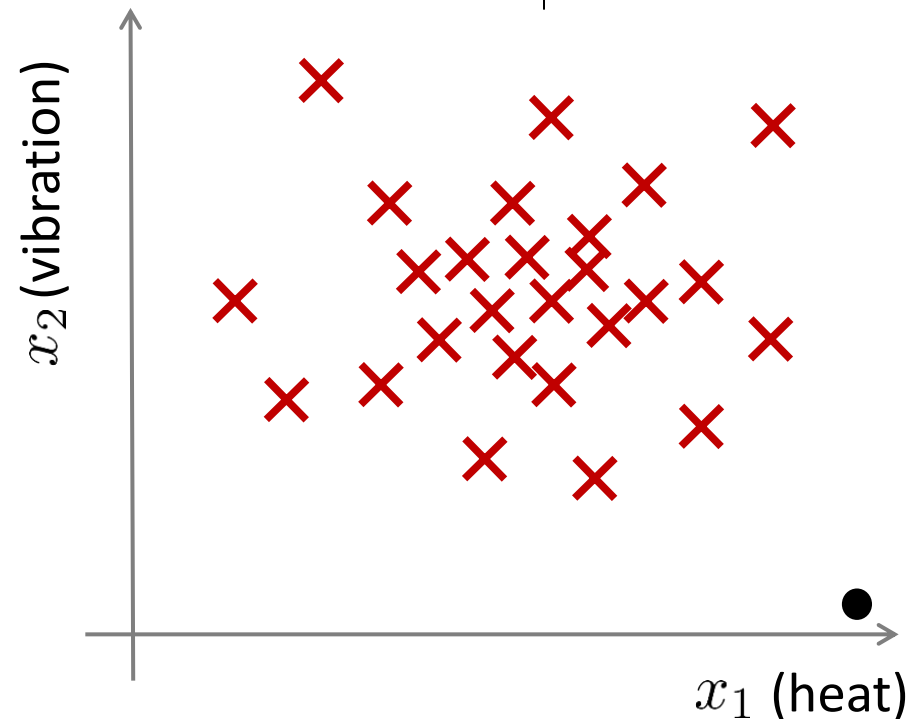
x_1 = heat generated

x_2 = vibration intensity

...

Dataset: $\{x^{(1)}, x^{(2)}, \dots, x^{(m)}\}$

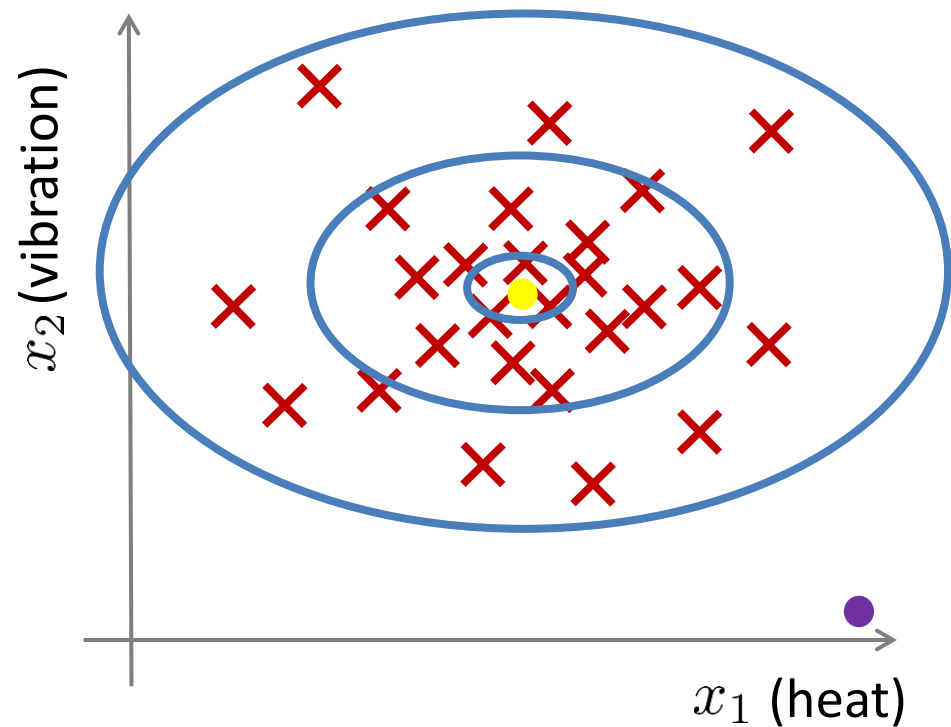
New engine: x_{test}



Density estimation

Dataset: $\{x^{(1)}, x^{(2)}, \dots, x^{(m)}\}$

Is x_{test} anomalous?



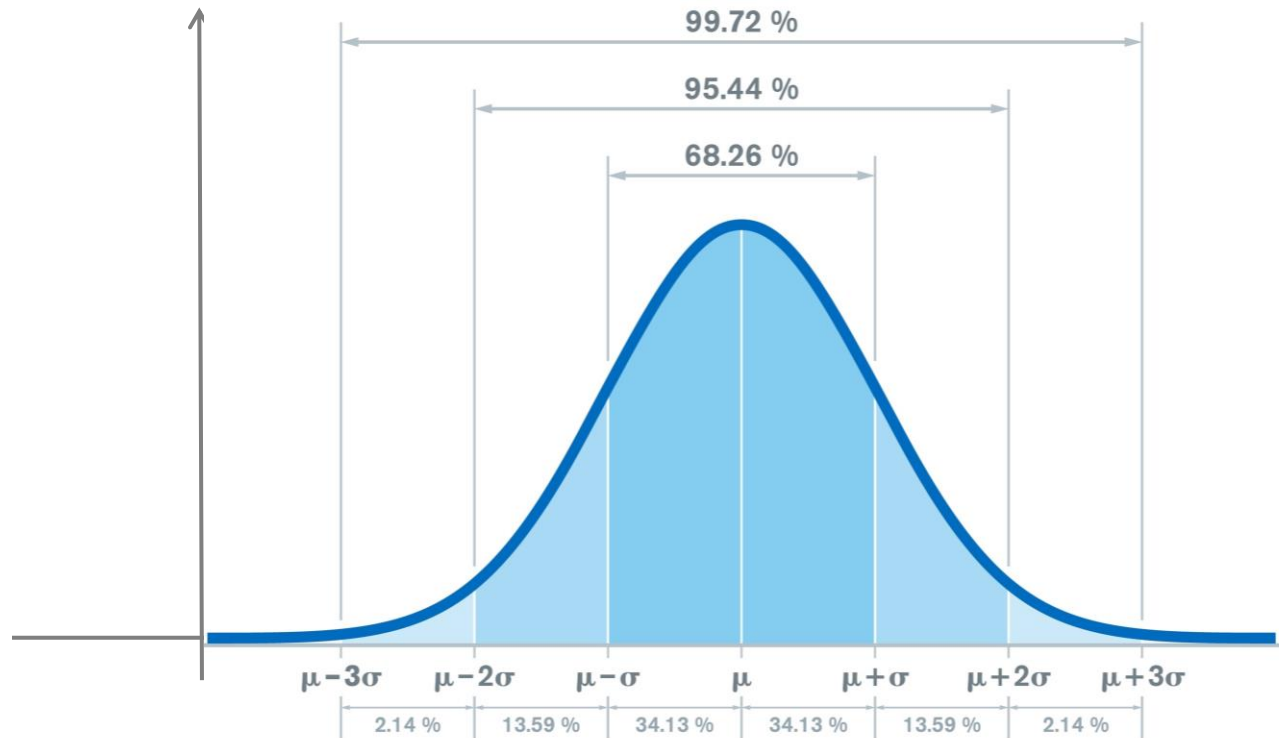
Model: $p(x)$

$p(x_{test}) < \epsilon$ (*flag anomaly*)

$p(x_{test}) \geq \epsilon$ (*ok*)

The Gaussian Model

Say $x \in \mathbb{R}$. If x is a distributed Gaussian with mean μ , variance σ^2 .



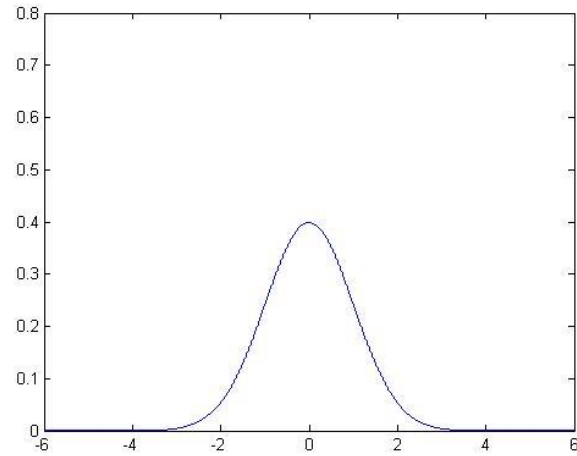
$$p(x; \mu, \sigma^2) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right)$$

$$\mu = \frac{1}{m} \sum_{i=1}^m x^{(i)}$$

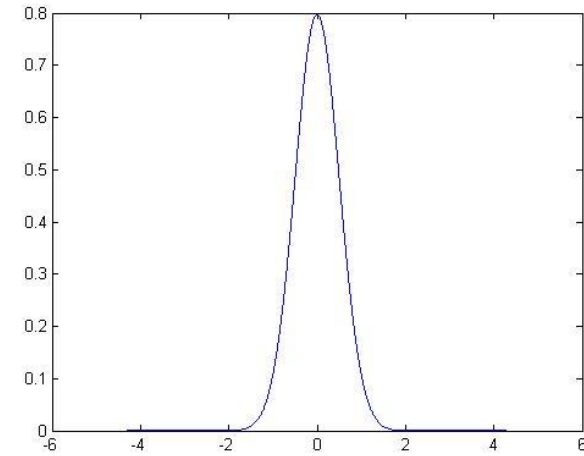
$$\sigma^2 = \frac{1}{m} \sum_{i=1}^m (x^{(i)} - \mu)^2$$

Gaussian distribution example

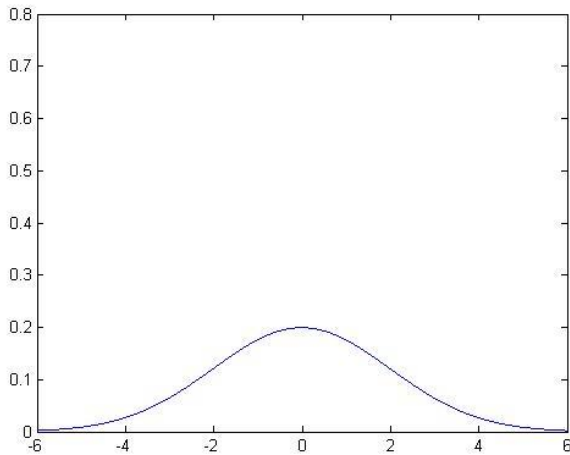
$$\mu = 0, \sigma = 1$$



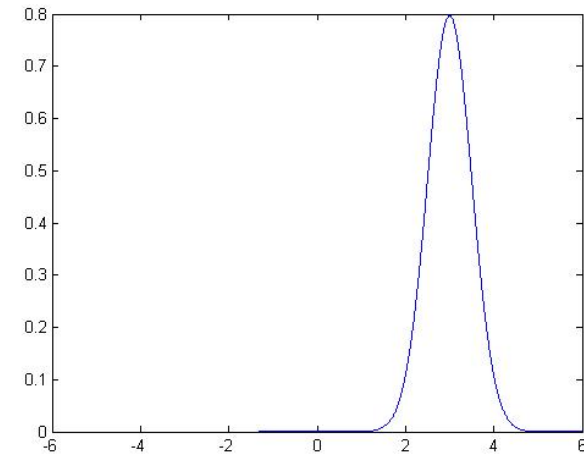
$$\mu = 0, \sigma = 0.5$$



$$\mu = 0, \sigma = 2$$



$$\mu = 3, \sigma = 0.5$$



The Gaussian Model

1. Choose features x_i that you think might be indicative of anomalous examples.
2. Fit parameters $\mu_1, \dots, \mu_n, \sigma_1^2, \dots, \sigma_n^2$

$$\mu_j = \frac{1}{m} \sum_{i=1}^m x_j^{(i)}$$

$$\sigma_j^2 = \frac{1}{m} \sum_{i=1}^m (x_j^{(i)} - \mu_j)^2$$

3. Given new example x , compute $p(x)$:

$$p(x) = \prod_{j=1}^n p(x_j; \mu_j, \sigma_j^2) = \prod_{j=1}^n \frac{1}{\sqrt{2\pi}\sigma_j} \exp\left(-\frac{(x_j - \mu_j)^2}{2\sigma_j^2}\right)$$

Anomaly if $p(x) < \varepsilon$

Example

- Imagine we are monitoring the response time (in milliseconds) of a web server. We have 4 "normal" observations:
 - Dataset (x): {10, 12, 11, 13} – single feature
- Calculate mean $\mu = \frac{10 + 12 + 11 + 13}{4} = \frac{46}{4} = 11.5$
- Calculate variance
 - $(10 - 11.5)^2 = (-1.5)^2 = 2.25$
 - $(12 - 11.5)^2 = (0.5)^2 = 0.25$
 - $(11 - 11.5)^2 = (-0.5)^2 = 0.25$
 - $(13 - 11.5)^2 = (1.5)^2 = 2.25$
 - Sum of squares = $2.25 + 0.25 + 0.25 + 2.25 = 5$ $\sigma^2 = \frac{5}{4} = 1.25$

Example

- Testing anomalies
 - A new request comes in with a response time of **20ms**.

$$p(x) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right)$$

1. The "Exponent" Part:

$$\frac{(20 - 11.5)^2}{2(1.25)} = \frac{8.5^2}{2.5} = \frac{72.25}{2.5} = 28.9$$

2. The Result:

$$p(20) = (\text{constant}) \cdot e^{-28.9}$$

Summary

- If we set our threshold $\epsilon = 0.01$:
 - $p(20) \approx 0 < 0.01 \rightarrow \text{ANOMALY}$ (The server is lagging significantly).
 - If a new point was 12ms,
 - $p(12)$ would be around 0.32, which is $> 0.01 \rightarrow \text{NORMAL}$.

Algorithm evaluation

Fit model $p(x)$ on training set $\{x^{(1)}, \dots, x^{(m)}\}$

On a cross validation/test example x , predict

$$y = \begin{cases} 1 & \text{if } p(x) < \varepsilon \text{ (anomaly)} \\ 0 & \text{if } p(x) \geq \varepsilon \text{ (normal)} \end{cases}$$

Possible evaluation metrics:

- True positive, false positive, false negative, true negative
- Precision/Recall
- F_1 -score

Can also use cross validation set to choose parameter ε

Anomaly detection

Very small number of positive examples ($y = 1$). (0-20 is common).

Large number of negative ($y = 0$) examples.

Many different “types” of anomalies. Hard for any algorithm to learn from positive examples what the anomalies look like; future anomalies may look nothing like any of the anomalous examples we’ve seen so far.

vs.

Supervised learning

Large number of positive and negative examples.

Enough positive examples for algorithm to get a sense of what positive examples are like, future positive examples likely to be similar to ones in training set.

Anomaly detection

- Fraud detection
- Manufacturing (e.g. aircraft engines)
- Monitoring machines in a data center

⋮

vs.

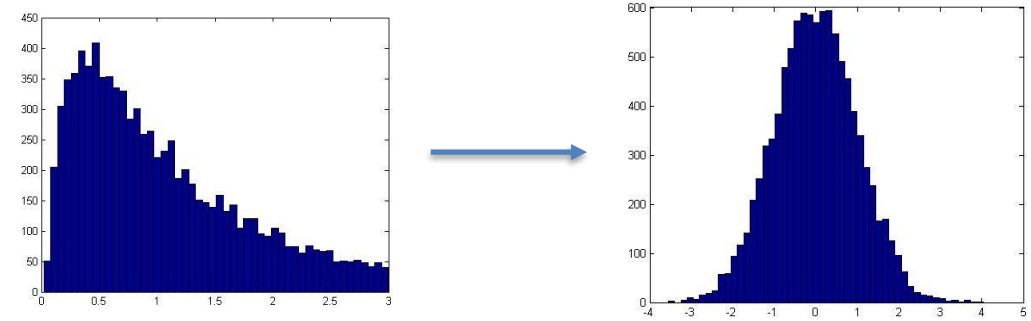
Supervised learning

- Email spam classification
- Weather prediction (sunny/rainy/etc).
- Cancer classification

⋮

How to choose features?

- Pick gaussian features
- For Non Gaussian features:
 - The Gaussian anomaly detection model assumes your features follow a bell curve. If a feature is highly skewed (e.g., all values are near 0 with a very long tail), the model may perform poorly.
 - 1: Plot a histogram of each feature. If it doesn't look like a bell curve, transform it.
 - 2: Apply mathematical transformations to make the distribution more symmetric:
 - Log transformation: $x_{new} = \log(x + c)$ (Most common for "long-tail" data).
 - Root transformation: $x_{new} = x^{\frac{1}{2}}$ or $x^{\frac{1}{3}}$.



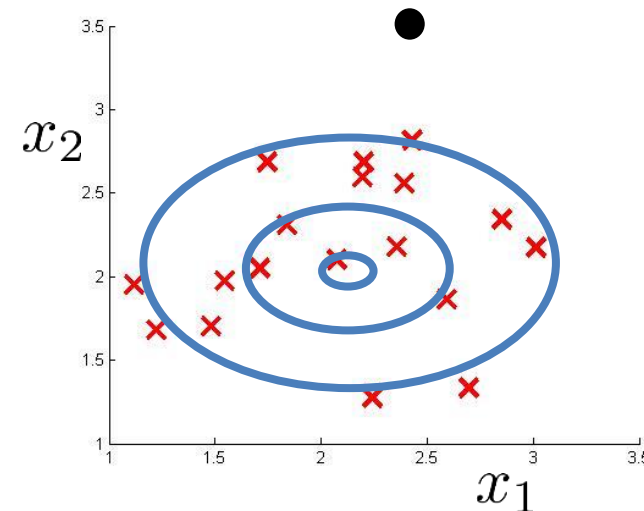
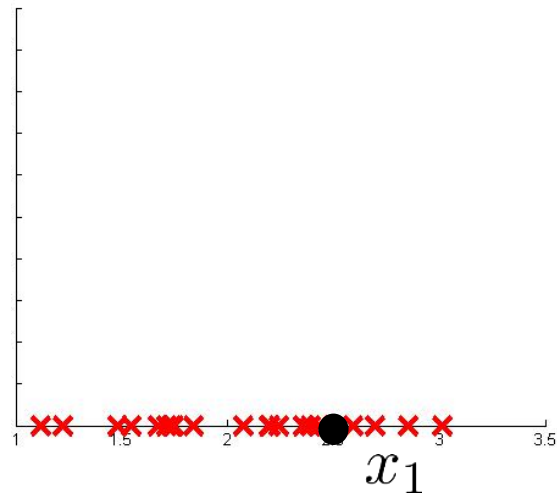
Error analysis for anomaly detection

Want $p(x)$ large for normal examples x .

$p(x)$ small for anomalous examples x .

Most common problem:

$p(x)$ is comparable (say, both large) for normal and anomalous examples



Differences

Feature	Gaussian Density Estimation (Ng)	Isolation Forest
Core Logic	"How unlikely is this point based on a distribution?"	"How easy is it to separate this point from others?"
Data Assumption	Assumes data follows a Gaussian (bell-shaped) curve.	No assumptions about the distribution shape.
Feature Scaling	Highly sensitive (requires log transforms for skewed data).	Not sensitive (based on tree splits).
Dimensionality	Can struggle with many features (Curse of Dimensionality).	Handles high-dimensional data very well.
Computation	Fast (simple mean/variance math).	Fast (efficient tree partitioning).
Best For	Small datasets, well-understood "normal" behavior.	Large, complex datasets with unknown distributions.